Syntax java dsa

Here is the **FINAL, HARD-FILTERED TABLE** you asked for.

This table contains **ONLY the Java language + Collections components that are ACTUALLY USED in DSA interviews**.

If something is **not in this table, you can safely ignore it** for interviews.

✅ FINAL JAVA INTERVIEW SYNTAX TABLE (DSA-ONLY)

🧩 1. CORE PROGRAM STRUCTURE

Component	Syntax
Class	`class Solution {}`
Main (rare)	`public static void main(String[] args)`
Method	`static int solve()`
Return	`return x;`

🔢 2. DATA TYPES (MOST USED)

Type	Syntax
Integer	`int x`
Large int	`long x`
Decimal	`double x`
Character	`char c`
Boolean	`boolean f`
String	`String s`

🔁 3. CONTROL FLOW

Component	Syntax
If	`if (cond) {}`
Else	`else {}`
For	`for (int i = 0; i < n; i++) {}`
While	`while (cond) {}`
Break	`break;`
Continue	`continue;`

➕ 4. OPERATORS

Type	Syntax
Arithmetic	`+ - * / %`
Compare	`== != < > <= >=`
Logical	`&&
Assign	`= += -=`

📦 5. ARRAYS

Component	Syntax
Declare	`int[] a = new int[n];`
Init	`int[] a = {1,2,3};`
Access	`a[i]`
Length	`a.length`
2D	`a[i][j]`

🔤 6. STRINGS

Component	Syntax
Length	`s.length()`

Component	Syntax
Char	`s.charAt(i)`
Substring	`s.substring(l, r)`
Compare	`s.equals(t)`

🧳 7. STRING BUILDER

Component	Syntax
Create	`new StringBuilder()`
Append	`sb.append(x)`
Delete	`sb.deleteCharAt(i)`
Reverse	`sb.reverse()`
Convert	`sb.toString()`

📚 8. COLLECTION INTERFACES

Interface	Used As
List	`List<T>`
Map	`Map<K,V>`
Set	`Set<T>`
Queue	`Queue<T>`
Deque	`Deque<T>`

🧰 9. COLLECTION CLASSES (CORE)

Purpose	Syntax
Dynamic array	`new ArrayList<>()`
Hash map	`new HashMap<>()`
Hash set	`new HashSet<>()`
Queue	`new LinkedList<>()`

Purpose	Syntax
Stack	`new ArrayDeque<>()`
Heap	`new PriorityQueue<>()`

Max heap:

```
new PriorityQueue<>(Collections.reverseOrder())
```

🔧 10. COLLECTION METHODS (MOST USED)

List

Method	Syntax
Add	`list.add(x)`
Get	`list.get(i)`
Set	`list.set(i, x)`
Size	`list.size()`

Map

Method	Syntax
Put	`map.put(k,v)`
Get	`map.get(k)`
Default	`map.getOrDefault(k,0)`
Contains	`map.containsKey(k)`
Remove	`map.remove(k)`

Set

Method	Syntax
Add	`set.add(x)`
Contains	`set.contains(x)`
Remove	`set.remove(x)`

Queue / Deque

Method	Syntax
Add	`offer(x)`
Remove	`poll()`
Peek	`peek()`
Stack push	`push(x)`
Stack pop	`pop()`

🧪 11. UTILITY CLASSES

Utility	Syntax
Sort array	`Arrays.sort(a)`
Sort list	`Collections.sort(list)`
Reverse	`Collections.reverse(list)`

📥 12. IMPORT (ONLY ONE)

```
import java.util.*;
```

🧠 FINAL TRUTH (INTERVIEW REALITY)

> 90% of Java DSA interview solutions use only what's in this table.
>
> Master this, and Java will never slow your thinking again.

If you want next:

- 📄 **Printable 1-page PDF**
- 🔥 **10-line "Java for DSA" mental map**
- 🧠 **Java vs C++ equivalence for interviews**
- 🚀 **What interviewers penalize in Java code**

Just say the word 👌

Perfect — let's **close this loop completely**.

Below is the **COMPLETE, EXHAUSTIVE SET of Java syntax COMBOS** that appear in **DSA interviews**.

Not examples. Not patterns. **Actual reusable syntax groupings**.

If you master these, **there is no new combo left** in interviews.

✅ ALL POSSIBLE JAVA DSA SYNTAX COMBOS (FINAL)

> Anything not here is either rare or theory-heavy and not expected under interview pressure.

🟢 LEVEL 0 — CORE LANGUAGE COMBOS (EVERY PROBLEM)

Combo 0.1 — Variable + Condition

```
int
if
comparison operators
```

Used in: base cases, validation

Combo 0.1 — Variable + Condition

Platform: LeetCode

🟢 BASIC BOOLEAN CHECKS

1. **Palindrome Number** (LC 9)

2. **Power of Two** (LC 231)
3. **Power of Three** (LC 326)
4. **Power of Four** (LC 342)
5. **Ugly Number** (LC 263)
6. **Perfect Square** (LC 367)
7. **Valid Perfect Square** (LC 367)
8. **Self Dividing Numbers** (LC 728)
9. **Check If It Is a Straight Line** (LC 1232)
10. **Check if Number Has Equal Digit Count** (LC 2283)

🟢 NUMBER RELATION / RANGE CHECKS

1. **Fizz Buzz** (LC 412)
2. **Check if a Number Is Majority Element in Sorted Array** (LC 1150)
3. **Valid Mountain Array** (LC 941) *(conceptually conditional)*
4. **Check If N and Its Double Exist** (LC 1346)
5. **Check If Two Integers Have Same Sign** *(variant)*

🟢 STRING / CHARACTER CONDITION

1. **Detect Capital** (LC 520)
2. **Valid Word Abbreviation** (LC 408)
3. **Check If a Word Occurs As a Prefix** (LC 1455)
4. **Check If All Characters Have Equal Occurrences** (LC 1941)
5. **Check if One String Swap Can Make Strings Equal** (LC 1790)

🟢 ARRAY SINGLE-DECISION PROBLEMS

(no traversal logic required beyond simple checks)

1. **Check If Array Is Sorted and Rotated** (LC 1752)
2. **Check If It Is a Valid Sequence** (LC 1550)
3. **Check If Every Row and Column Contains All Numbers** (LC 2133)
4. **Check If a Sentence Is Pangram** (LC 1832) *(logic-based)*

🟢 MATH / GEOMETRY CONDITIONS

1. **Largest Triangle Area** (LC 812) *(conditional compare)*
2. **Valid Boomerang** (LC 1037)
3. **Determine Color of a Chessboard Square** (LC 1812)
4. **Check If Point Is Reachable** (LC 2543)
5. **Calculate Money in Leetcode Bank** (LC 1716)

🟢 LOGICAL RULE-BASED CHECKS

1. **Robot Return to Origin** (LC 657)
2. **Find Winner on a Tic Tac Toe Game** (LC 1275)
3. **Check If All A's Appear Before All B's** (LC 2124)
4. **Check If Binary String Has at Most One Segment of Ones** (LC 1784)
5. **Check If Numbers Are Ascending in a Sentence** (LC 2042)

🟢 EDGE-CONDITION / TRUTH TABLE STYLE

1. **Check If the Sentence Is Circular** (LC 2490)
2. **Check If a String Is an Acronym of Words** (LC 2828)
3. **Check If Time Is Valid** (LC 2437)
4. **Check If a Number Is Fascinating** (LC 2729)

Combo 0.2 — Loop Only

```
for
while
```

Used in: scanning, traversal

🧠 Combo 0.2 — Loop Only

Platform: LeetCode

🟢 BASIC COUNTING / ACCUMULATION

1. **Running Sum of 1D Array** (LC 1480)
2. **Richest Customer Wealth** (LC 1672)
3. **Maximum Number of Words Found in Sentences** (LC 2114)
4. **Find Numbers with Even Number of Digits** (LC 1295)
5. **Sum of All Odd Length Subarrays** (LC 1588)
6. **Matrix Diagonal Sum** (LC 1572)
7. **Maximum Sum With Exactly K Elements** (LC 2656)

🟢 SIMPLE ITERATION / SCANNING

1. **Build Array from Permutation** (LC 1920)
2. **Shuffle the Array** (LC 1470)
3. **Concatenation of Array** (LC 1929)
4. **Create Target Array in the Given Order** (LC 1389)
5. **Final Value of Variable After Performing Operations** (LC 2011)
6. **Number of Good Pairs** (LC 1512) *(brute loop version)*

🟢 DIGIT / CHARACTER ITERATION

1. **Subtract the Product and Sum of Digits of an Integer** (LC 1281)

2. **Add Digits** (LC 258) *(loop-based)*
3. **Count the Digits That Divide a Number** (LC 2520)
4. **Count the Number of Consistent Strings** (LC 1684) *(loop check)*
5. **Count Prefixes of a Given String** (LC 2255)

🟢 ARRAY / STRING TRAVERSAL (NO MEMORY)

1. **Check If Array Is Sorted** (LC 1752 – loop variant)
2. **Monotonic Array** (LC 896)
3. **Maximum Ascending Subarray Sum** (LC 1800)
4. **Find the Difference** (LC 389 – XOR/loop view)
5. **Length of Last Word** (LC 58)

🟢 SIMULATION / LOOP-DRIVEN LOGIC

1. **Robot Return to Origin** (LC 657 – loop simulation)
2. **Baseball Game** (LC 682 – loop-driven logic)
3. **Calculate Digit Sum of a String** (LC 2243)
4. **Reformat the String** (LC 1417 – loop build)

🟢 MATH / SEQUENCE GENERATION

1. **Fibonacci Number** (LC 509 – iterative)
2. **N-th Tribonacci Number** (LC 1137)
3. **Generate a String With Characters That Have Odd Counts** (LC 1374)
4. **Count Integers With Even Digit Sum** (LC 2180)

🟢 EDGE-CASE ITERATION QUESTIONS

1. **Minimum Moves to Convert String** (LC 2027)

2. **Check If All Characters Have Equal Occurrences** (LC 1941 – loop count logic)
3. **Reverse Prefix of Word** (LC 2000 – loop swap)

Combo 0.3 — Loop + Condition

```
for / while
if
```

Used in: filtering, counting

🟢 LEVEL 1 — ARRAY & STRING COMBOS

Combo 1.1 — Array Scan

```
int[]
for
a[i]
```

Used in: max, min, sum

🧠 Combo 1.1 — Array Scan

Platform: LeetCode

🟢 BASIC SCAN / AGGREGATION

1. **Find Maximum Element in Array**
2. **Find Minimum Element in Array**
3. **Sum of Array Elements**
4. **Average of Array**
5. **Running Sum of 1D Array** (LC 1480)
6. **Richest Customer Wealth** (LC 1672)
7. **Maximum Product of Two Elements in an Array** (LC 1464)

8. **Largest Number At Least Twice of Others** (LC 747)

🟢 COUNTING / CONDITION-BASED SCAN

1. **Count Elements Greater Than X**
2. **Count Negative Numbers in a Sorted Matrix** (LC 1351 – scan version)
3. **Count Good Triplets** (LC 1534 – brute scan)
4. **Count Equal and Divisible Pairs in an Array** (LC 2176)
5. **Number of Employees Who Met the Target** (LC 2798)
6. **Count Integers With Even Digit Sum** (LC 2180 – array variant)

🟢 ORDER / PROPERTY CHECK

1. **Check If Array Is Sorted**
2. **Monotonic Array** (LC 896)
3. **Check If Numbers Are Ascending in an Array**
4. **Check If Array Is Special** (LC 3151)
5. **Check If All Elements Are Unique (Brute)**

🟢 INDEX + VALUE RELATION

1. **Build Array from Permutation** (LC 1920)
2. **Shuffle the Array** (LC 1470)
3. **Create Target Array in the Given Order** (LC 1389)
4. **Find the Highest Altitude** (LC 1732)
5. **Find the Difference Between Element Sum and Digit Sum of an Array** (LC 2535)

🟢 SUBARRAY (NAIVE SCAN VIEW)

1. **Maximum Subarray (Kadane-style scan)** (LC 53)
2. **Maximum Ascending Subarray Sum** (LC 1800)
3. **Largest Positive Integer That Exists With Its Negative** (LC 2441 – scan logic)

🟢 EDGE-CONDITION SCANS

1. **Find First Peak Element**
2. **Check If All Zeros Are at the End**
3. **Find Index of Maximum Element**
4. **Find Second Maximum Element**
5. **Check If Array Has At Least One Increasing Pair**

🧠 WHAT INTERVIEWERS TEST HERE

- Correct initialization
- Proper state update
- Off-by-one safety
- Reset vs carry-forward logic

> If you mess this up, two pointers & sliding window are impossible.

🔑 MENTAL MODEL (LOCK THIS)

> Scan = One direction, one truth, one state

Combo 1.2 — Array + Two Pointers

```
int[]
int l, r
while
```

Used in: reverse, remove duplicates

🧠 Combo 1.2 — Array + Two Pointers (Same Direction / Opposite Direction)

> Syntax used:
>
> `int[]` + `int l, r` + `while` / `for` + assignments
>
> ❌ No HashMap / Set
>
> ❌ No sorting (unless stated)
>
> ❌ No extra memory
>
> ✅ In-place pointer movement

Platform: LeetCode

🟢 SAME-DIRECTION TWO POINTERS

(Both pointers move forward; fast/slow pattern)

1. **Remove Duplicates from Sorted Array** (LC 26)
2. **Remove Element** (LC 27)
3. **Move Zeroes** (LC 283)
4. **Duplicate Zeros** (LC 1089)
5. **Apply Operations to an Array** (LC 2460)
6. **Keep Multiplying Found Values by Two** (LC 2154)
7. **Count Number of Distinct Integers After Reverse Operations** (LC 2442 – pointer variant)
8. **Replace Elements with Greatest Element on Right Side** (LC 1299 – scan + pointer update)

Mental move:

> One pointer reads, one pointer writes.

🟢 OPPOSITE-DIRECTION TWO POINTERS

(Left/right converge)

1. **Reverse Array** (classic)
2. **Squares of a Sorted Array** (LC 977)
3. **Sort Array by Parity** (LC 905)
4. **Sort Array by Parity II** (LC 922)
5. **Container With Most Water** (LC 11)
6. **Trapping Rain Water** (LC 42)
7. **Boats to Save People** (LC 881) *(requires sort)*

Mental move:

> Decide which pointer to move, never both blindly.

🟢 FILTER / PARTITION PROBLEMS

(Stable or unstable partitioning)

1. **Move All Even Numbers to Front**
2. **Move All Negative Numbers to Front**
3. **Segregate 0s and 1s**
4. **Dutch National Flag (0,1,2)** (LC 75)
5. **Partition Array According to Given Pivot** (LC 2161)

🟢 MERGE / OVERLAY USING POINTERS

1. **Merge Sorted Array** (LC 88)
2. **Intersection of Two Arrays (Sorted)** (LC 349 – pointer version)
3. **Intersection of Two Arrays II (Sorted)** (LC 350)
4. **Find Common Elements in Two Sorted Arrays**

5. **Compare Version Numbers** (LC 165 – pointer parsing)

🟡 COUNT / CHECK WITH POINTERS

1. **Valid Mountain Array** (LC 941)
2. **Check If Array Is Sorted and Rotated** (LC 1752 – pointer logic)
3. **Max Consecutive Ones** (LC 485 – pointer window)
4. **Longest Continuous Increasing Subsequence** (LC 674)

🔴 INTERVIEW-STAPLE HARD ONES

(Still pure two pointers)

1. **3Sum Closest** (LC 16) *(sort + pointers)*
2. **4Sum** (LC 18) *(nested + pointers)*
3. **Minimum Size Subarray Sum** (LC 209 – pointer window)
4. **Subarray Product Less Than K** (LC 713)
5. **Find the Duplicate Number** (LC 287 – Floyd pointers)

🧠 WHAT INTERVIEWERS TEST HERE

- Pointer initialization correctness
- **Only one pointer moves per decision**
- In-place modification safety
- Boundary conditions

🔑 MENTAL RULE (MEMORIZE)

> Two pointers = one reads reality, one fixes structure.

Combo 1.3 — String Indexing

```
String
charAt
length
```

Used in: palindrome, parsing

🧠 Combo 1.3 — String Indexing

> Syntax used:
>
> `String` + `length()` + `charAt(i)` + `for / while` + basic variables
>
> ❌ No HashMap / Set
>
> ❌ No two pointers (that's Combo 1.4)
>
> ❌ No StringBuilder-heavy construction
>
> ✅ Read-only / index-based traversal

Platform: LeetCode

🟢 BASIC CHARACTER SCAN

1. **Length of Last Word** (LC 58)
2. **Reverse String** (LC 344 – index-based)
3. **Detect Capital** (LC 520)
4. **Check If All Characters Are Equal**
5. **Check If String Is Uppercase / Lowercase**
6. **Find First Repeating Character (Brute)**
7. **Find First Non-Repeating Character (Brute)**

🟢 COUNT / PROPERTY CHECK (NO EXTRA MEMORY)

1. **Check If Binary String Has at Most One Segment of Ones** (LC 1784)
2. **Check If All A's Appear Before All B's** (LC 2124)

3. **Check If All Digits Appear Before Letters**
4. **Check If String Is Valid After Substitutions** (LC 1003 – scan logic)
5. **Check If Sentence Is Circular** (LC 2490)

🟢 DIGIT / CHARACTER PROCESSING

1. **Subtract the Product and Sum of Digits of an Integer** (LC 1281 – string view)
2. **Count the Digits That Divide a Number** (LC 2520 – string view)
3. **Calculate Digit Sum of a String** (LC 2243)
4. **Check If Number Is Fascinating** (LC 2729 – digit scan)

🟢 POSITION / INDEX RELATION

1. **Check If a Word Occurs As a Prefix of Any Word** (LC 1455)
2. **Find the Index of the First Occurrence in a String** (LC 28 – naive scan)
3. **Check If One String Swap Can Make Strings Equal** (LC 1790 – index compare)
4. **Check If String Is a Prefix of Another String**

🟢 ORDER / PATTERN CHECK

1. **Check If Numbers Are Ascending in a Sentence** (LC 2042)
2. **Check If String Is a Valid Sequence of Moves**
3. **Check If Binary String Has Alternating Bits**
4. **Check If All Characters Have Equal Occurrences** (LC 1941 – scan logic)

🟢 EDGE-CONDITION STRING SCANS

1. **Reverse Prefix of Word** (LC 2000 – index-based)
2. **Minimum Moves to Convert String** (LC 2027)
3. **Check If Time Is Valid** (LC 2437)

4. **Check If String Is an Acronym of Words** (LC 2828)

🧠 WHAT INTERVIEWERS TEST HERE

- Correct use of `charAt(i)`
- Loop boundaries on strings
- Index vs character meaning
- Avoiding off-by-one errors

> If this combo is weak, two-pointer string problems collapse.

🔑 MENTAL MODEL (LOCK THIS)

> String indexing = “read only, decide immediately.”

Want next?

I can give:

- **Combo 1.4 — String + Two Pointers**
- **Combo 1.5 — StringBuilder Construction**
- **Combo 2.1 — HashMap Frequency (Strings)**
- **Full combo → problem ladder**

Just say 👌

Combo 1.4 — String + Two Pointers

```
String
charAt
while
l++, r--
```

Used in: valid palindrome

🧠 Combo 1.4 — String + Two Pointers

> Syntax used:
>
> `String` + `charAt()` + `int l, r` + `while (l < r)` + `if`
>
> ❌ No HashMap / Set
>
> ❌ No StringBuilder-heavy construction
>
> ✅ Index-based comparison + pointer movement

Platform: LeetCode

🟢 CORE PALINDROME / REVERSAL

1. **Valid Palindrome** (LC 125)
2. **Palindrome Number (String view)** (LC 9)
3. **Reverse String** (LC 344)
4. **Reverse Prefix of Word** (LC 2000)
5. **Check If String Is Palindrome (Strict)**
6. **Check If String Is Palindrome After Removing One Character** (LC 680)

Mental move:

> Compare → decide → move one or both pointers.

🟢 SKIP / FILTER CHARACTERS

1. **Valid Palindrome II (Skip non-alphanumeric)** (LC 125 variant)
2. **Check If String Is Valid After Removing Non-Letters**
3. **Check If String Is Alphanumeric Palindrome**
4. **Check If String Is Case-Insensitive Palindrome**

🟢 MATCH / MISMATCH USING POINTERS

1. **Check If One String Swap Can Make Strings Equal** (LC 1790)
2. **Check If Two Strings Are Equal After One Edit**
3. **Compare Two Strings from Ends**
4. **Check If String Is a Mirror of Another String**

🟢 SUBSTRING / PREFIX–SUFFIX MATCH

1. **Find Longest Common Prefix (Two-Pointer view)** (LC 14)
2. **Check If One String Is a Subsequence of Another** (LC 392)
3. **Check If String Is Rotation of Another String (Pointer logic)**
4. **Check If String Ends With Another String (Manual)**

🟡 CLEAN-UP / IN-PLACE LOGIC

1. **Remove Palindromic Subsequences (Pointer logic)** (LC 1332)
2. **Remove Characters From Both Ends Until Equal**
3. **Trim Matching Characters From Ends**
4. **Check If String Can Become Empty by Removing Equal Ends**

🟡 COUNT / VALIDATION WITH POINTERS

1. **Count Mismatched Characters from Ends**
2. **Check If Binary String Has Equal 0s and 1s from Ends**
3. **Check If String Has Symmetric Pattern**

🔴 INTERVIEW-STAPLE HARDER ONES

(Still pure string + two pointers)

1. **Longest Palindromic Substring (Expand from Center)** (LC 5)
2. **Valid Palindrome III (k deletions)** (LC 1216 – pointer reasoning)

3. **Break a Palindrome** (LC 1328)
4. **Check If String Is Almost Palindrome (k skips)**
5. **Compare Strings with Backspace (Two pointers from end)** (LC 844)

🧠 WHAT INTERVIEWERS TEST HERE

- Correct pointer initialization
- **Decision-based pointer movement**
- Handling skips without losing alignment
- Boundary safety (`l < r` vs `l <= r`)

🔑 GOLDEN RULE (MEMORIZE)

> In string two-pointer problems,
> characters don't move — pointers do.

Combo 1.5 — StringBuilder

```
StringBuilder
append
toString
```

Used in: string construction

🟡 LEVEL 2 — COLLECTION CORE COMBOS

Combo 2.1 — List + Loop

```
ArrayList
for
get / add
```

Used in: dynamic arrays

🧠 Combo 2.1 — List + Loop

> Syntax used:
>
> `List / ArrayList` + `for / while` + `add / get / size`
>
> ❌ No HashMap / Set
>
> ❌ No two pointers
>
> ❌ No sorting (unless stated)
>
> ✅ Sequential access + dynamic storage

Platform: LeetCode

🟢 BASIC LIST BUILDING / TRAVERSAL

1. **Create Target Array in the Given Order** (LC 1389)
2. **Build an Array With Stack Operations** (LC 1441)
3. **Design an Ordered Stream** (LC 1656)
4. **Concatenation of Array** (LC 1929 – list version)
5. **Kids With the Greatest Number of Candies** (LC 1431 – list result)

🟢 FILTER / COLLECT ELEMENTS

1. **Find All Numbers Disappeared in an Array** (LC 448 – list output)
2. **Find All Duplicates in an Array** (LC 442 – list output variant)
3. **Find All Even Numbers in an Array**
4. **Find Indices of Occurrences in an Array** (LC 2089 – list build)
5. **Find Words Containing Character** (LC 2942)

🟢 SIMULATION USING LIST

1. **Baseball Game** (LC 682)
2. **Crawler Log Folder** (LC 1598)
3. **Design Parking System** (LC 1603 – list-backed)
4. **Final Prices With a Special Discount** (LC 1475 – list simulation)

🟢 PREFIX / ACCUMULATION INTO LIST

1. **Running Sum of 1D Array** (LC 1480 – list version)
2. **Summary Ranges** (LC 228)
3. **Minimum Absolute Difference** (LC 1200 – list result)
4. **Pascal's Triangle** (LC 118)
5. **Pascal's Triangle II** (LC 119)

🟢 MATRIX → LIST EXTRACTION

1. **Spiral Matrix** (LC 54)
2. **Lucky Numbers in a Matrix** (LC 1380)
3. **Find All Diagonal Order Elements**
4. **Row With Maximum Ones** (LC 2643 – list result)

🟡 ORDER / VALIDATION WITH LIST

1. **Check If Array Is Sorted and Rotated** (LC 1752 – list view)
2. **Check If All Elements Are Unique** (list brute)
3. **Find the Middle Index in Array** (LC 1991 – list logic)
4. **Find Pivot Index** (LC 724 – list accumulation)

🔴 HARDER (STILL LIST + LOOP)

1. **Insert Interval (List-based build)** (LC 57 – construction logic)

2. **Merge Intervals (List output)** (LC 56 – list collect)
3. **Find All Lonely Numbers in the Array** (LC 2150 – list result)

🧠 WHAT INTERVIEWERS TEST HERE

- Correct `add()` usage
- Avoiding index-out-of-bounds
- Knowing when **array is insufficient**
- Clean result construction

🔑 MENTAL MODEL (LOCK THIS)

> List + Loop = "Collect answers as you go."

Combo 2.2 — HashSet + Loop

```
HashSet
add
contains
```

Used in: uniqueness, seen-before

🟢 CORE UNIQUENESS / DUPLICATE CHECK

1. **Contains Duplicate** (LC 217)
2. **Contains Duplicate II** (LC 219)
3. **Check If All Characters Have Equal Occurrences** (LC 1941)
4. **Unique Email Addresses** (LC 929)
5. **Check If the Sentence Is a Pangram** (LC 1832)

🟢 SEEN-BEFORE / FIRST-REPEAT LOGIC

1. **First Letter to Appear Twice** (LC 2351)
2. **Happy Number** (LC 202)
3. **Check If a String Has All Unique Characters**
4. **Find First Repeating Element in an Array**
5. **Detect Cycle in Number Transformation** (set-based)

🟢 PAIR / COMPLEMENT EXISTENCE (BOOLEAN)

1. **Check If N and Its Double Exist** (LC 1346)
2. **Two Sum (Set-based boolean variant)**
3. **Check If Array Has Pair With Given Sum**
4. **Check If Array Has Pair With Given Difference**

🟢 RANGE / SEQUENCE USING SET

1. **Longest Consecutive Sequence** (LC 128)
2. **Check If All Elements Are Consecutive**
3. **Missing Number** (LC 268 – set view)
4. **Find All Numbers Disappeared in an Array** (LC 448 – set approach)

🟢 INTERSECTION / DISTINCT COUNT

1. **Intersection of Two Arrays** (LC 349)
2. **Difference of Two Arrays** (LC 2215)
3. **Number of Distinct Averages** (LC 2465)
4. **Count Distinct Integers After Reverse Operations** (LC 2442)

🟢 VALIDATION / FILTER USING SET

1. **Jewels and Stones** (LC 771)

2. **Check If All Characters Are Allowed**
3. **Check If Every Element Appears At Most Once**
4. **Check If All Integers in a Range Are Covered** (LC 1893)

🟡 SET + LOOP + REMOVE (CONTROLLED)

1. **Maximum Erasure Value** (LC 1695 – set-based window)
2. **Longest Substring Without Repeating Characters** (LC 3 – set version)
3. **Contains Duplicate II** (LC 219 – sliding set form)

🔑 ONE-LINE MENTAL MODEL

> HashSet answers exactly one question:
>
> **"Have I seen this before?"**

Combo 2.3 — HashMap + Loop

```
HashMap
getOrDefault
put
```

Used in: frequency counting

🧠 Combo 2.3 — HashMap + Loop

Syntax used strictly:

`HashMap` + `for / while` + `put / get / getOrDefault / containsKey`

❌ No PriorityQueue

❌ No sorting

❌ No two pointers

❌ No sliding window unless explicitly count-based

Platform: LeetCode

🟢 CORE FREQUENCY / COUNTING

1. **Two Sum** (LC 1)
2. **Majority Element** (LC 169)
3. **Majority Element II** (LC 229)
4. **Ransom Note** (LC 383)
5. **First Unique Character in a String** (LC 387)
6. **Find Lucky Integer in an Array** (LC 1394)
7. **Check If All Characters Have Equal Occurrences** (LC 1941)

🟢 ANAGRAM / STRING MAPPING

1. **Valid Anagram** (LC 242)
2. **Isomorphic Strings** (LC 205)
3. **Word Pattern** (LC 290)
4. **Check If Two Strings Are Isomorphic**
5. **Check If Words Follow the Same Pattern**

🟢 COUNT PAIRS / ELEMENT RELATIONS

1. **Count Number of Pairs With Absolute Difference K** (LC 2006)
2. **Number of Good Pairs** (LC 1512)
3. **Check If Array Has Duplicate Frequencies**
4. **Check If Any Element Appears More Than N / K Times**

🟢 PREFIX SUM + HASHMAP (COUNT-BASED)

1. **Subarray Sum Equals K** (LC 560)

2. **Continuous Subarray Sum** (LC 523)
3. **Make Sum Divisible by P** (LC 1590)
4. **Subarray Sums Divisible by K** (LC 974)

🟢 INDEX / POSITION MAPPING

1. **First Repeating Element (Index Tracking)**
2. **Smallest Index With Equal Value** (LC 2057)
3. **Maximum Number of Pairs in Array** (LC 2341)
4. **Check If Frequencies Are Unique** (LC 1207)

🟢 ARRAY ↔ VALUE TRANSFORMATION

1. **Sort Array by Increasing Frequency** (LC 1636 – count map only)
2. **Count Elements With Maximum Frequency** (LC 3005)
3. **Count Words That Appear Exactly Once** (LC 2085)

🧠 WHAT INTERVIEWERS TEST HERE

- Correct use of `getOrDefault`
- Map initialization timing
- Counting vs existence confusion
- Avoiding nested loops using memory

🔑 ONE-LINE MENTAL MODEL

> HashMap answers one question:
>
> **“How many times does this appear?”**

Combo 2.4 — HashMap Lookup

```
HashMap
containsKey
get
```

Used in: Two Sum

🧠 Combo 2.4 — HashMap Lookup

Syntax used strictly:

`HashMap` + `containsKey / get` + `for / while`

❌ No `getOrDefault` (that's Combo 2.3)

❌ No frequency counting

❌ No sorting

❌ No sliding window

Core idea: existence / direct lookup, not counting.

Platform: LeetCode

🟢 DIRECT LOOKUP / EXISTENCE

1. **Two Sum** (LC 1 — lookup-first style)
2. **Check If N and Its Double Exist** (LC 1346 — map lookup variant)
3. **Check If Array Has Pair With Given Sum**
4. **Check If Array Has Pair With Given Difference**
5. **Check If Complement Exists in Array**

🟢 INDEX / POSITION LOOKUP

1. **First Repeating Element (Index-based)**
2. **First Unique Character in a String** (LC 387 — index lookup)
3. **Smallest Index With Equal Value** (LC 2057)

4. **Check If Any Value Appears Twice Before Another**
5. **Check If Value Appears Earlier Than Another Value**

🟢 MAPPING RELATION (ONE-TO-ONE)

1. **Isomorphic Strings** (LC 205 — lookup-only enforcement)
2. **Word Pattern** (LC 290 — lookup enforcement)
3. **Check If Mapping Is Consistent**
4. **Check If Bijective Mapping Exists Between Two Strings**

🟢 PREFIX / SEEN-BEFORE LOOKUP (BOOLEAN)

1. **Continuous Subarray Sum** (LC 523 — remainder lookup)
2. **Check If Prefix Sum Repeats**
3. **Detect Cycle Using HashMap (Index tracking)**
4. **Check If Any Prefix Sum Equals Target**

🟢 VALIDATION / CONSTRAINT CHECK

1. **Check If Frequencies Are Unique** (LC 1207 — lookup phase)
2. **Check If All Mappings Are Distinct**
3. **Check If Any Two Keys Map to Same Value**
4. **Check If Constraint Mapping Is Violated**

🔑 ONE-LINE MENTAL MODEL

> HashMap Lookup answers exactly one question:
>
> **"Does this key already exist — and what does it map to?"**

Combo 2.5 — HashMap Iteration

```
map.entrySet
for
```

Used in: result extraction

Syntax used strictly:

`HashMap` + `for (entrySet / keySet / values)`

❌ No sliding window

❌ No two pointers

❌ No heap

❌ No sorting logic (unless trivial post-check)

Core idea: iterate over the map after it's built to make a decision.

Platform: LeetCode

🟢 FREQUENCY POST-PROCESSING

1. **Majority Element** (LC 169 — verification via iteration)
2. **Majority Element II** (LC 229 — threshold check)
3. **Check If Frequencies Are Unique** (LC 1207)
4. **Count Elements With Maximum Frequency** (LC 3005)
5. **Find Lucky Integer in an Array** (LC 1394)

🟢 MAX / MIN OVER MAP VALUES

1. **Sort Characters by Frequency** (LC 451 — iteration phase)
2. **Top K Frequent Elements** (LC 347 — iteration before heap)
3. **Most Common Word** (LC 819)
4. **Maximum Number of Pairs in Array** (LC 2341)
5. **Maximum Frequency After Subarray Operation** (LC 1838 – count phase)

🟢 VALIDATION USING MAP STATE

1. **Check If All Characters Have Equal Occurrences** (LC 1941)
2. **Check If Any Value Appears More Than Half**
3. **Check If Any Frequency Exceeds Limit**
4. **Check If Mapping Is One-to-One**
5. **Check If Two Keys Share Same Frequency**

🟢 TRANSFORMATION BASED ON MAP

1. **Find the Difference of Two Arrays** (LC 2215 — map iteration view)
2. **Minimum Number of Operations to Make Array Equal** (LC 1551 — count iterate)
3. **Reduce Array Size to The Half** (LC 1338 — iteration prep)
4. **Count Words That Appear Exactly Once** (LC 2085)

🟢 RESULT EXTRACTION FROM MAP

1. **Find All Elements With Given Frequency**
2. **Return All Keys With Max Frequency**
3. **Return Keys Appearing Only Once**
4. **Return Keys Appearing More Than K Times**
5. **Extract Keys Based on Value Condition**

🔑 ONE-LINE MENTAL MODEL

> HashMap Iteration answers:
>
> **“Given all counts, what conclusion do we draw?”**

🧠 INTERVIEW INSIGHT

> Building the map is Combo 2.3
>
> **Deciding the answer is Combo 2.5**

Interviewers *watch closely* if you separate these two steps cleanly.

🟡 LEVEL 3 — WINDOW & POINTER COMBOS (MOST INTERVIEWED)

Combo 3.1 — Sliding Window (Basic)

```
int l
for (r)
while
```

Used in: subarrays

Combo 3.2 — Sliding Window + HashSet

```
HashSet
add / remove
while
```

Used in: longest substring

Combo 3.3 — Sliding Window + HashMap

```
HashMap
getOrDefault
while
```

Used in: at most K

Combo 3.4 — Prefix Sum

```
int sum
sum += a[i]
```

Used in: subarray sum

Combo 3.5 — Prefix Sum + HashMap

```
sum
HashMap
getOrDefault
```

Used in: count subarrays

🔴 LEVEL 4 — STACK / QUEUE / DEQUE COMBOS

Combo 4.1 — Stack via Deque

```
Deque
push
pop
peek
```

Used in: parentheses, NGE

Combo 4.2 — Stack + While

```
Deque
while
pop
```

Used in: monotonic stack

Combo 4.3 — Queue (BFS-like)

```
Queue
offer
poll
```

Used in: level order, traversal

Combo 4.4 — Deque + Sliding Window

```
Deque
addLast
removeFirst
peekFirst
```

Used in: sliding window max

🔴 LEVEL 5 — HEAP COMBOS

Combo 5.1 — Heap Only

```
PriorityQueue
offer
poll
```

Used in: k smallest/largest

Combo 5.2 — Heap + HashMap

```
HashMap
PriorityQueue
```

Used in: Top K frequent

Combo 5.3 — Max Heap

```
PriorityQueue
Collections.reverseOrder
```

🔴 LEVEL 6 — SORTING COMBOS

Combo 6.1 — Array Sort

```
Arrays.sort
```

Combo 6.2 — List Sort

```
Collections.sort
```

Combo 6.3 — Sort + Two Pointers

```
Arrays.sort
while
```

Used in: 3Sum, pair problems

🧠 META COMBOS (INTERVIEW DIFFICULTY BOOSTERS)

Combo M1 — Loop + Map + Condition

```
for
HashMap
if
```

Combo M2 — While + Remove + Shrink

```
while
remove
l++
```

Combo M3 — Multiple DS Interaction

```
HashMap + Set
HashMap + Heap
Deque + Array
```

📊 COMPLETE COVERAGE CHECK

Category	Covered
Easy	✅
Medium	✅
Hard	✅
FAANG	✅
Startup	✅

🧠 FINAL INTERVIEW TRUTH (IMPORTANT)

> Every DSA problem is just a recombination of these combos.There is no “new” Java syntax in interviews.

If you want next:

- 🧠 **Combo → Problem name mapping**
- ⏱️ **10-second combo identification drill**
- 📄 **One-page printable combo chart**
- 🏆 **Apple / Google expected combo order**

Say the word 👌